Requirements to Enhance Holos Visualization

Goal of Holos' visualization:

Overlay one or more Holos' results on a map both in a single time instance and in a time series, using public domain free python packages.

Current Visualization and Technology Used

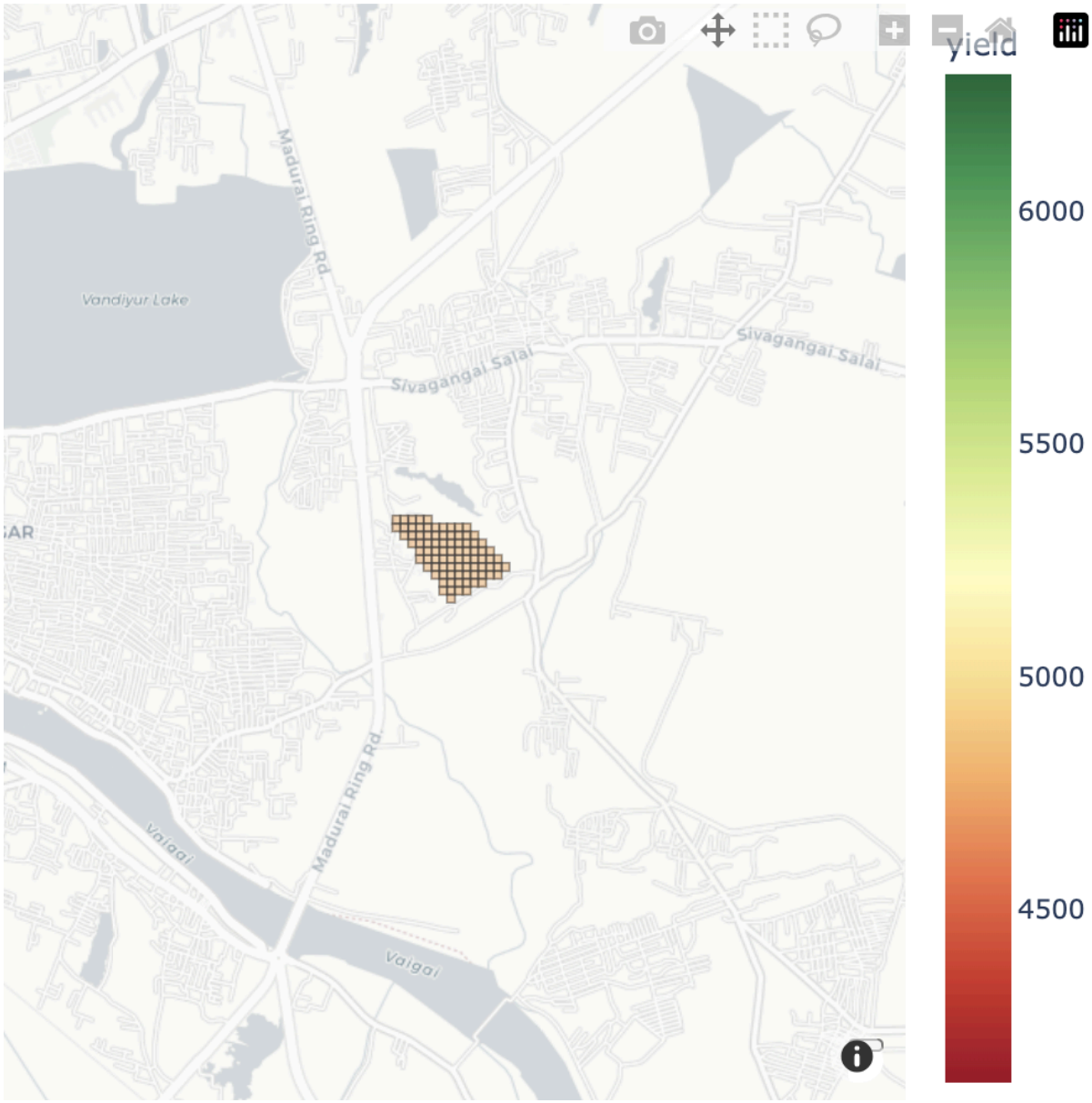
1. Content: Overlaying of forecasted yield on a map.
2. Python modules used: plotly and plotly.express (geopandas is a choice, but needs to be evaluated and Holos uses pandas 3.5 while geopandas may need a higher version of pandas, requiring the platform to upgrade). Code snippet is in document and in github. Data to the rendering function is a subset of the simulated region to fit the region drawn on the map. For the project with UTA, either inputs to the rendering function or the subset function can be provided

```
#Get farm geojson from input json
farm_geo = holos_ip['user_input_json']['Blocks'][0]['ROIs'][0]['ROIConfiguration']['geoJSON']['features'][0]['geometry']

#Alternative to get to farm_geo
with open('TanjoreCompKuruvaiRice_RedBlocks.json') as f:
    d = json.load(f)
farm_geo = d['Blocks'][0]['ROIs'][0]['ROIConfiguration']['geoJSON']['features'][0]['geometry']

#Create a Polygon object of the farm
farm_polygon, poly_center = get_farm_poly_center(farm_geo)
#Create a subset of the data for cultivar_of_choic and sub-blocks that are within farm_geo
#summary_output_df is the summary xls in a dataframe format.
subset_df, var_min, var_max = create_geo_var_subset (summary_output_df, farm_polygon, opt_cultivar, 'yield')
#Print yield on map
map_fname = render_subset_on_map (dir, subset_df, poly_center, all_blocks_json_data, opt_cultivar, var_min,
var_max)
chart_fnames.append(os.path.join(dir, map_fname))
```

3. Sample overlay of yield of a particular variety of rice over a map. File is generated as a html.

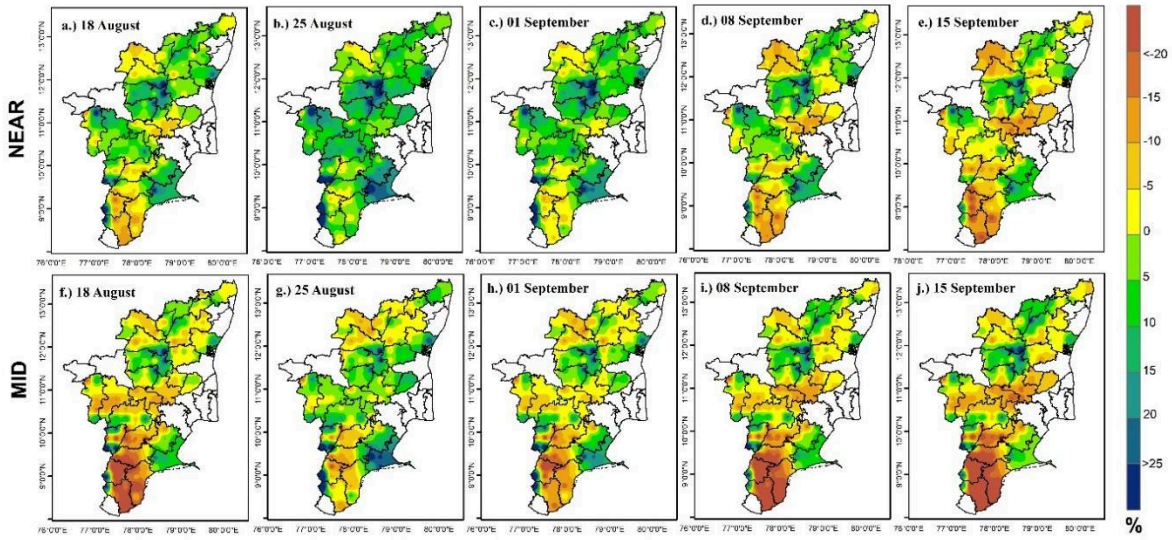


a. Each small square (sub-block/field in farming lingo) is colored based on the yield it produces. The colormap is based on a larger output set, and is likely incorrect for the specific cultivar.
b. If one mouses over on the sub-block, the co-ordinates and yield will be displayed.

Enhancements to be made

1. Is this the best map to overlay a Holos simulated output?
2. The boundaries around the sub-blocks should be removed.

3. Achieve visualizations of the following kind - boundaries seem to align with map's district (sub-regional?) boundaries, and multiple colors within those sub-regions. Eg., consider each of the founded regions in the following maps to be districts. The districts could be broken down into smaller sub-blocks in Holos' terms. The boundaries of the sub-blocks within a district are not shown (unlike the current Holos rendering), but the summ'ed up yield for a group of sub-blocks is displayed within the district.



impact of climate change on sorghum yield under different sowing dates

4. Develop a time-series visualization. Data for this cam be obtained from the summary output as it has multiple years' simulations.
5. Overlaying different simulation outputs for a composite view: yield, yield/water productivity, yield/nitrogen productivity. Sub-regions where each is these is highest should receive highest scores. Weightage based views?

Code

Create Polygon object

```
def get_farm_poly_center(farm_geo):
    # Create a polygon object from farm_geo
    farm_polygon = shape(farm_geo)
    # Polygon(polygon)
    poly_center = list(Polygon(farm_polygon).centroid.coords)
    return farm_polygon, poly_center
```

Sub set code

```
def create_geo_var_subset (df, polygon, cult, var):
    #This may cause some edge drops
    if var == 'yield':
        col_to_use = 'Unadjusted Yield (kg/ha))'
    elif var == 'co2':
        col_to_use = 'CO2 Emitted (kg/ha)'
    elif var == 'n2':
        col_to_use = 'Nitrogen Used (kg/ha)'
    elif var == 'water':
        col_to_use = 'Water Used (mm)'
    df = df.astype({'Center Lat': float, 'Center Long': float})
    yield_min = df[col_to_use].min()
    yield_max = df[col_to_use].max()
    cult_df = df[(df['Cultivar'] == cult)]
    cult_unique_ll = cult_df.groupby(['Sub Block ID']).mean(['Unadjusted Yield (kg/ha))']).reset_index()
    # Drop all those rows (sub-blocks) whose center points do not lie within the farm polygon - this may have some
edge issues
    for index, row in cult_unique_ll.iterrows():
        point = Point(row['Center Long'], row['Center Lat'])
        if not polygon.contains(point):
            cult_unique_ll.drop([index], inplace=True)
    cult_unique_ll.rename(columns={"Unadjusted Yield (kg/ha))": "yield", "Sub Block ID": "blockID"}, inplace=True)
    cult_unique_ll.rename(columns={"Center Lat": "lat", "Center Long": "lon"}, inplace=True)
    cult_unique_ll.reset_index(inplace=True)
    print(yield_min, yield_max)
    return cult_unique_ll, yield_min, yield_max
```

Rendering code:

```
def render_subset_on_map(dir, cult_unique_ll, holos_sub_blocks_geo_json, cult, var_min, var_max):
    # Works with a nice location based map; zoom, center and range_color work, but sorta blandish
    # Zoom may need to be dynamic based on number of sub-blocks or so
    # Need geojson
    fig = px.choropleth_mapbox(cult_unique_ll, geojson=holos_sub_blocks_geo_json,
                               locations=cult_unique_ll['blockID'].astype(str), color="yield",
                               featureidkey='properties.id',
                               color_continuous_scale="RdYlGn",
                               range_color=[var_min, var_max],
                               mapbox_style="carto-positron",
                               zoom=12,
                               # center = {'lon':-121, 'lat':38},
                               center={'lon': poly_center[0][0], 'lat': poly_center[0][1]},
                               width=500,
                               height=500,
                               opacity=0.5
                               )
    fig.update_layout(margin={"r": 0, "t": 0, "l": 0, "b": 0})
    fname = os.path.join(dir, "integrated_street_map_yield_overlay_" + cult + ".html")
    plotly.offline.plot(fig, include_plotlyjs='cdn', filename=fname, auto_open=True)
```

```
return fname
```